MSSQL SELECT語法結構

MSSQL select 語法結構

在 Microsoft SQL Server（MSSQL）中，**SELECT** 是最基本也最常用的查詢語法，用來從資料表中讀取資料。

下面用「結構 → 範例 → 補充」方式，幫你一次整理清楚：

一、SELECT 基本語法結構

```sql
SELECT 欄位1, 欄位2, ...
FROM 資料表名稱
WHERE 條件
GROUP BY 欄位
HAVING 條件
ORDER BY 欄位 [ASC | DESC];
```

二、各區塊說明（重點）

1️⃣ SELECT（選擇欄位）

```sql
SELECT name, age
```

👉 指定要顯示的欄位
👉 * 代表全部欄位

```sql
SELECT *
```

2️⃣ FROM（資料來源）

```sql
FROM Users
```

👉 指定資料表

3️⃣ WHERE（條件篩選）

```sql
WHERE age > 18
```

👉 在資料讀出前篩選

常用運算：

- = 等於
- <> 不等於

- `> < >= <=`
- `LIKE` 模糊查詢
- `IN` 範圍
- `BETWEEN`

4️⃣ GROUP BY（分組）

```SQL
GROUP BY department
```

👉 通常搭配聚合函數使用（如 `COUNT`, `SUM`）

5️⃣ HAVING（分組後條件）

```SQL
HAVING COUNT(*) > 5
```

👉 和 WHERE 差別：

- WHERE：分組前
- HAVING：分組後

6️⃣ ORDER BY（排序）

```SQL
ORDER BY age DESC
```

👉 排序方式：

- `ASC`：由小到大（預設）
- `DESC`：由大到小

三、完整範例

```SQL
SELECT department, COUNT(*) AS 人數
FROM Employees
WHERE age > 25
GROUP BY department
HAVING COUNT(*) > 3
ORDER BY 人數 DESC;
```

👉 意思是：

1. 找出 25 歲以上員工
2. 依部門分組
3. 計算每個部門人數
4. 只保留人數 > 3 的部門

5. 依人數由大到小排序

四、執行順序（非常重要🔥）

雖然寫法是這樣：

```SQL
SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY
```

👉 **實際執行順序是：**

```
FROM
→ WHERE
→ GROUP BY
→ HAVING
→ SELECT
→ ORDER BY
```

👉 這是 SQL 初學者最容易搞錯的地方！

五、常見補充

✔️ 欄位別名（Alias）

```SQL
SELECT name AS 姓名
```

✔️ 去除重複

```SQL
SELECT DISTINCT city
```

✔️ TOP（取前幾筆，MSSQL 特有）

```SQL
SELECT TOP 10 *
FROM Users
```

六、一句話總結

👉 **SELECT** 就是：

👉 **「從哪裡（FROM）抓資料 → 篩選（WHERE）→ 分組（GROUP BY）→ 排序（ORDER BY）」**

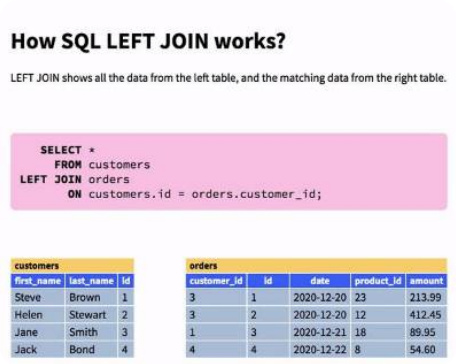
如果你要，我可以幫你整理一份「SQL 常考語法總表（適合考試/論文/工作用）」或做成一頁速查表 👍

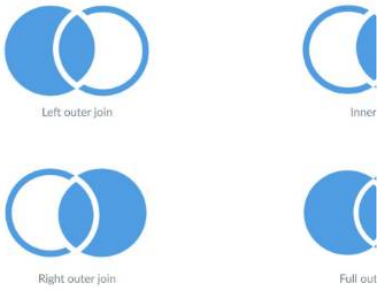
二. 請補充+範例說明 left join 及 inner join 的不同
又select 時, 不指明 left join 及 inner join , 直接使用 where a.欄位=b.欄位, 此時是使用什麼 join呢.

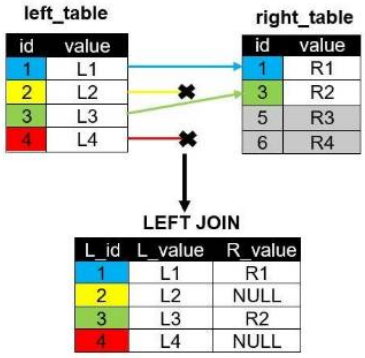
這題很重要，尤其在資料分析與論文實作中（像你在做醫療影像資料整合時也會用到）。我用「概念＋圖像理解＋SQL範例」幫你一次搞懂：

一、LEFT JOIN vs INNER JOIN（核心差異）

🔵 LEFT JOIN（左連接）







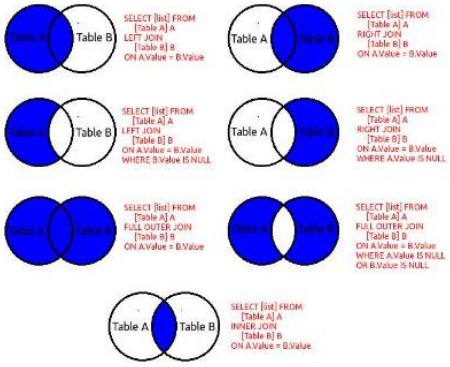
👉 保留左表全部資料

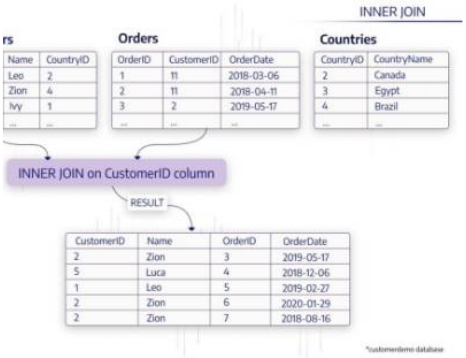
```
SQL
SELECT *
FROM A
LEFT JOIN B ON A.id = B.id;
```

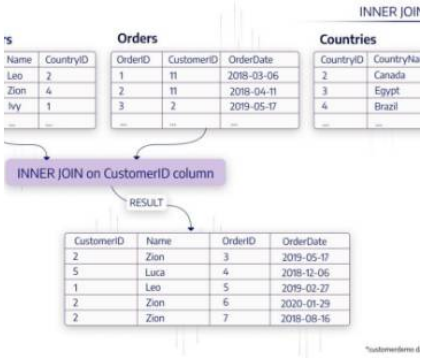
📌 特性：

- A（左表）全部保留
- B（右表）沒有對應 → 顯示 `NULL`

🔴 INNER JOIN（內連接）







👉 只保留「兩邊都有對應」的資料

```
SQL
SELECT *
FROM A
INNER JOIN B ON A.id = B.id;
```

📌 特性：

- 只有 A 和 B **都有匹配的資料才會出現**
- 沒配對 → 直接消失

二、用實際資料範例說明

假設兩張表：

表 A（學生）

id	name
1	小明
2	小華
3	小美

表 B（成績）

id	score
1	90
3	80
4	70

🔴 INNER JOIN 結果

```SQL
SELECT A.name, B.score
FROM A
INNER JOIN B ON A.id = B.id;
```

👉 結果：

name	score
小明	90
小美	80

📌 小華（id=2）被排除（因為沒有成績）

🔵 LEFT JOIN 結果

```sql
SELECT A.name, B.score
FROM A
LEFT JOIN B ON A.id = B.id;
```

👉 結果：

name	score
小明	90
小華	NULL
小美	80

📌 小華保留，但 score = NULL

三、最關鍵差異一句話

👉 **INNER JOIN：交集**
👉 **LEFT JOIN：左表全部 + 右表有就接上**

四、重點問題（你問的第2題🔥）

❓ 不寫 JOIN，直接用 WHERE

```sql
SELECT *
FROM A, B
WHERE A.id = B.id;
```

👉 這是什麼 JOIN？

✔️ 答案：👉 等同於 INNER JOIN

📌 原因：

```sql
FROM A, B   → 先做「Cartesian Product（笛卡兒積）」
WHERE A.id = B.id → 再過濾
```

👉 最終效果 =
👉 **只留下符合條件的資料 → INNER JOIN**

五、對照整理（考試必背🔥）

寫法	類型
`INNER JOIN`	內連接

寫法	類型
`LEFT JOIN`	左連接
`FROM A, B WHERE A.id = B.id`	✅ INNER JOIN
沒條件的 `FROM A, B`	❌ 笛卡兒積（全部組合）

六、實務重要提醒（很關鍵⚠️）

❗ LEFT JOIN + WHERE 會「變質」

```sql
SELECT *
FROM A
LEFT JOIN B ON A.id = B.id
WHERE B.score > 80;
```

👉 ⚠️ 這樣會變成：

👉 **效果接近 INNER JOIN**

因為：

- `B.score > 80` 會把 `NULL` 刪掉

✔️ 正確寫法（保留 LEFT JOIN 特性）

```sql
SELECT *
FROM A
LEFT JOIN B ON A.id = B.id AND B.score > 80;
```

七、總結一句話（超重要）

👉

- `INNER JOIN`：只要有配對
- `LEFT JOIN`：左邊全留
- `WHERE A.id = B.id`：👉 本質是 INNER JOIN

如果你要，我可以幫你整理：
✅ JOIN 全種類（RIGHT / FULL / CROSS）
✅ 或幫你出幾題 SQL 練習（偏考試或實務）